AIRLINE RESERVATION & CHECK-IN – SPECIFICATION DOCUMENT

PROBLEM STATEMENT:

Design and implement a Java console application for an Airline Reservation and Check-in System that manages flights, aircrafts, seats, passengers, bookings, payments, and boarding passes.
The system must follow object-oriented principles such as encapsulation, inheritance, and aggregation, and it should prevent double booking.The system will allow passengers to book flights, make payments, check in, cancel bookings, and issue boarding passes

CLASS REQUIREMENTS:

The application should include the following main classes:

1. **Flight** – stores flight number, origin, destination, schedule, and assigned aircraft.
2. **Aircraft** – represents an airplane with a unique ID, model, capacity, and seats.
3. **Seat** – defines seat number and availability status.
4. **Passenger** – stores passenger ID, name, and contact details.
5. **Booking** – links passenger, flight, seat, and payment information.
6. **Payment** – records fare, taxes, total amount, and payment status.
7. **BoardingPass** – generated after successful check-in; shows passenger, flight, seat, gate, and timestamp

BUSINESS RULES:

1. A seat can be booked only if it is available for the selected flight.
2. Payment confirmation is mandatory before generating a boarding pass.
3. Cancellations should immediately free up the seat according to airline policy.
4. Check-in assigns seats and issues boarding passes.
5. Every booking must have a direct link to one flight, one passenger, and one seat.
6. The system must prevent double booking of any seat.
7. Encapsulation should be applied to secure all class attributes

CONSOLE INTERFACE REQUIREMENTS:

The application must be menu-driven, with the following options:

- Add Flight
- Add Passenger
- Create Booking
- Make/Confirm Payment
- Check-in & Issue Boarding Pass
- Cancel Booking
- Display Flights
- Exit

ADDITIONAL REQUIREMENTS:

- Validate all user inputs (e.g., IDs, flight numbers, seat availability).
- Follow OOP principles strictly.
- Provide clear console messages for successful and failed operations.

EXPECTED OUTPUT BEHAVIOR:

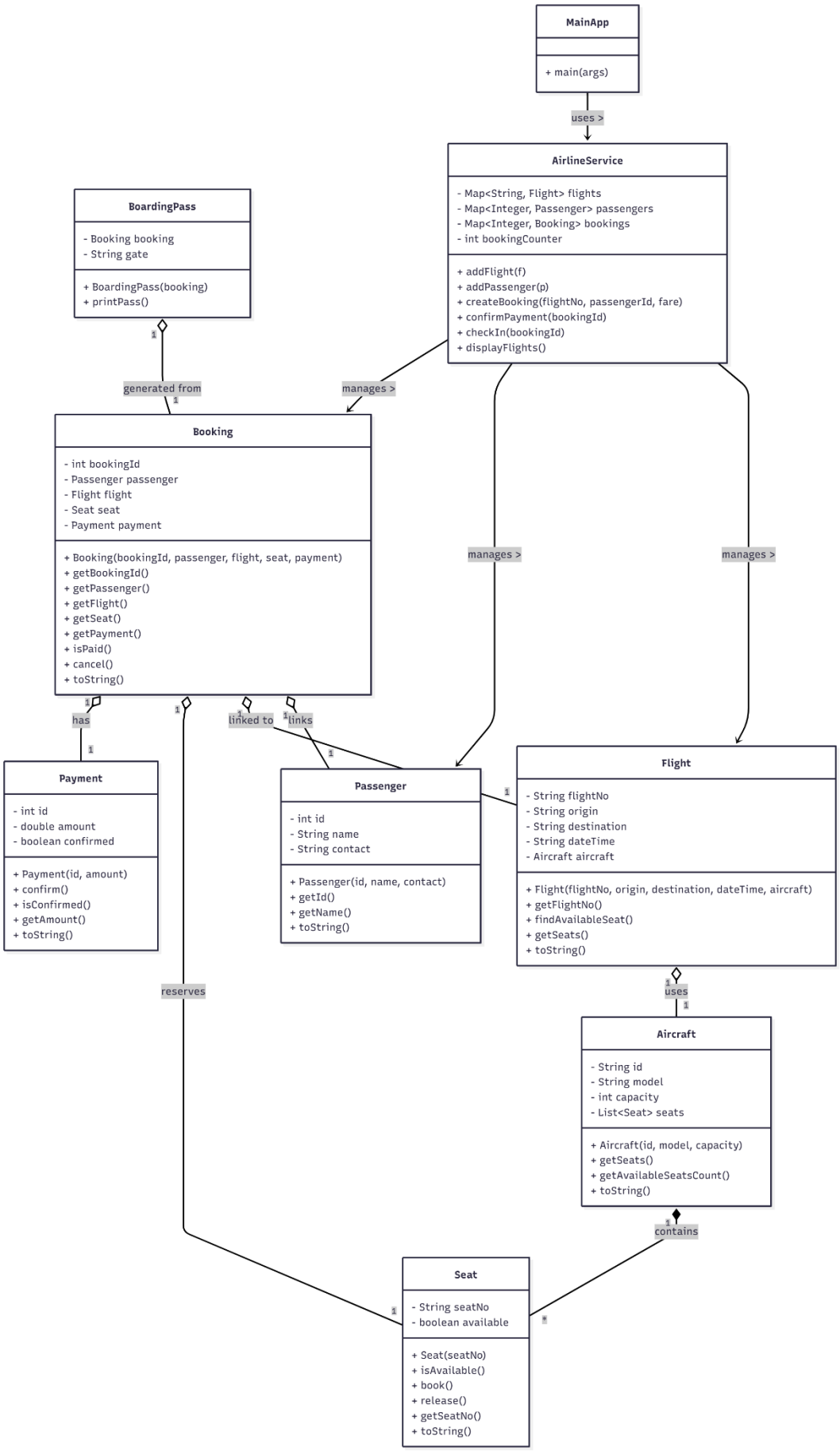
1. Display a booking receipt showing flight details, seat number, fare, taxes, and total amount.
2. Show a boarding pass with passenger details, flight no., seat no., gate, and date/time stamp.
3. Update and display seat availability after each booking, check-in, or cancellation.
4. Prevent duplicate bookings and invalid payments through validation messages.

QUESTIONS FOR STUDENTS:

1. Draw the UML Class Diagram for the above system.
2. Implement the Classes with necessary Data Members and Methods for System and Business Rules.
3. Use Aggregation, Inheritance and Polymorphism wherever required.

4. Implement the main method for Menu Driven System.Implement the main method for a menu-driven system.

❖ **DRAW THE UML CLASS DIAGRAM FOR THE SYSTEM:**



- **IMPLEMENT THE CLASSES WITH NECESSARY DATA MEMBERS AND METHODS FOR SYSTEM AND BUSINESS RULES:**

PROJECT STRUCTURE:

com.airline.model

```
├── Flight.java
├── Aircraft.java
├── Seat.java
├── Passenger.java
├── Booking.java
├── Payment.java
├── BoardingPass.java
```

com.airline.service

```
├── AirlineService.java
```

com.airline.main

```
├── MainApp.java
```

PACKAGE : com.airline.model

Flight.java

```
package com.airline.model;
import java.util.*;
public class Flight {
    private String flightNo;
    private String origin;
    private String destination;
    private String dateTime;
    private Aircraft aircraft;
    public Flight(String flightNo, String origin, String destination, String dateTime, Aircraft aircraft) {
        this.flightNo = flightNo;
        this.origin = origin;
        this.destination = destination;
        this.dateTime = dateTime;
        this.aircraft = aircraft;
    }
    public String getFlightNo() { return flightNo; }
    public Seat findAvailableSeat() {
        for (Seat s : aircraft.getSeats()) {
            if (s.isAvailable()) return s;
        }
        return null;
    }
    public List<Seat> getSeats() {
        return aircraft.getSeats();
    }
    @Override
    public String toString() {
        return flightNo + " | " + origin + " → " + destination + " | " + dateTime + " | Seats: " +
aircraft.getAvailableSeatsCount();
    }
}
```

Aircraft.java

```
package com.airline.model;
import java.util.*;
public class Aircraft {
    private String id;
    private String model;
    private int capacity;
    private List<Seat> seats;
    public Aircraft(String id, String model, int capacity) {
        this.id = id;
        this.model = model;
        this.capacity = capacity;
        this.seats = new ArrayList<>();
        for (int i = 1; i <= capacity; i++) {
            seats.add(new Seat("S" + i));
        }
    }
    public List<Seat> getSeats() {
        return seats;
    }
    public int getAvailableSeatsCount() {
        int count = 0;
        for (Seat s : seats)
            if (s.isAvailable()) count++;
        return count;
    }
    @Override
    public String toString() {
        return id + " - " + model + " (Capacity: " + capacity + ")";
    }
}
```

Seat.java

```
package com.airline.model;
public class Seat {
    private String seatNo;
    private boolean available = true;

    public Seat(String seatNo) {
        this.seatNo = seatNo;
    }
    public boolean isAvailable() {
        return available;
    }
    public void book() {
        available = false;
    }
    public void release() {
        available = true;
    }
    public String getSeatNo() {
        return seatNo;
    }
    @Override
```

```
    public String toString() {
        return seatNo + " (" + (available ? "Available" : "Booked") + ")";
    }
}
```

Passenger.java

```
package com.airline.model;
public class Passenger {
    private int id;
    private String name;
    private String contact;
    public Passenger(int id, String name, String contact) {
        this.id = id;
        this.name = name;
        this.contact = contact;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    @Override
    public String toString() {
        return id + " - " + name + " (" + contact + ")";
    }
}
```

Booking.java

```
package com.airline.model;
public class Booking {
    private int bookingId;
    private Passenger passenger;
    private Flight flight;
    private Seat seat;
    private Payment payment;
    public Booking(int bookingId, Passenger passenger, Flight flight, Seat seat, Payment payment) {
        this.bookingId = bookingId;
        this.passenger = passenger;
        this.flight = flight;
        this.seat = seat;
        this.payment = payment;
    }
    public int getBookingId() { return bookingId; }
    public Passenger getPassenger() { return passenger; }
    public Flight getFlight() { return flight; }
    public Seat getSeat() { return seat; }
    public Payment getPayment() { return payment; }
    public boolean isPaid() {
        return payment.isConfirmed();
    }
    public void cancel() {
        seat.release();
        System.out.println("Booking cancelled. Seat " + seat.getSeatNo() + " released.");
    }
    @Override
    public String toString() {
        return "Booking #" + bookingId + " | " + passenger.getName() + " | " + flight.getFlightNo() + " | Seat: "
+ seat.getSeatNo();
```

```
    }
}
```

Payment.java

```
package com.airline.model;
public class Payment {
    private int id;
    private double amount;
    private boolean confirmed;
    public Payment(int id, double amount) {
        this.id = id;
        this.amount = amount;
        this.confirmed = false;
    }
    public void confirm() {
        confirmed = true;
    }
    public boolean isConfirmed() {
        return confirmed;
    }

    public double getAmount() { return amount; }
    @Override
    public String toString() {
        return "Payment ID: " + id + " | Amount: ₹" + amount + " | Status: " + (confirmed ? "Confirmed" :
"Pending");
    }
}
```

BoardingPass.java

```
package com.airline.model;
public class BoardingPass {
    private Booking booking;
    private String gate = "A1";
    public BoardingPass(Booking booking) {
        this.booking = booking;
    }
    public void printPass() {
        System.out.println("\n===== BOARDING PASS =====");
        System.out.println("Passenger: " + booking.getPassenger().getName());
        System.out.println("Flight: " + booking.getFlight().getFlightNo());
        System.out.println("Seat: " + booking.getSeat().getSeatNo());
        System.out.println("Gate: " + gate);
        System.out.println("Status: CHECKED-IN");
        System.out.println("==========================\n");
    }
}
```

PACKAGE : com.airline.service

AirlineService.java

```
package com.airline.service;
import com.airline.model.*;
import java.util.*;
public class AirlineService {
    private Map<String, Flight> flights = new HashMap<>();
```

```
    private Map<Integer, Passenger> passengers = new HashMap<>();
    private Map<Integer, Booking> bookings = new HashMap<>();
    private int bookingCounter = 1;
    public void addFlight(Flight f) {
        flights.put(f.getFlightNo(), f);
        System.out.println("Flight added successfully: " + f);
    }
    public void addPassenger(Passenger p) {
        passengers.put(p.getId(), p);
        System.out.println("Passenger added successfully: " + p);
    }
    public void createBooking(String flightNo, int passengerId, double fare) {
        Flight f = flights.get(flightNo);
        Passenger p = passengers.get(passengerId);
        if (f == null || p == null) {
            System.out.println("Invalid flight or passenger!");
            return;
        }
        Seat seat = f.findAvailableSeat();
        if (seat == null) {
            System.out.println("No available seats!");
            return;
        }
        seat.book();
        Payment payment = new Payment(bookingCounter, fare);
        Booking booking = new Booking(bookingCounter++, p, f, seat, payment);
        bookings.put(booking.getBookingId(), booking);

        System.out.println("Booking created successfully!");
        System.out.println(booking);
        System.out.println(payment);
    }
    public void confirmPayment(int bookingId) {
        Booking b = bookings.get(bookingId);
        if (b == null) {
            System.out.println("Booking not found!");
            return;
        }
        b.getPayment().confirm();
        System.out.println("Payment confirmed for Booking #" + bookingId);
    }
    public void checkIn(int bookingId) {
        Booking b = bookings.get(bookingId);
        if (b == null) {
            System.out.println("Booking not found!");
            return;
        }
        if (!b.isPaid()) {
            System.out.println("Payment not confirmed!");
            return;
        }
        BoardingPass bp = new BoardingPass(b);
        bp.printPass();
    }
    public void displayFlights() {
```

```
        System.out.println("\n===== FLIGHT LIST =====");
        for (Flight f : flights.values()) {
            System.out.println(f);
        }
    }
}
```

❖ USE ENCAPSULATION, INHERITANCE, AGGREGATION, AND POLYMORPHISM WHEREVER REQUIRED:

ENCAPSULATION (DATA HIDING)
DEFINITION:

Encapsulation means keeping the data (attributes) private and exposing only necessary methods (getters/setters or specific behaviors).

In your code:

- All data members (like seatNo, available, flightNo, amount, etc.) are private.
- They are accessed and modified only via public methods.

Example:

```
public class Seat {
    private String seatNo;
    private boolean available = true;
    public Seat(String seatNo) { this.seatNo = seatNo; }
    public boolean isAvailable() { return available; }
    public void book() { available = false; }
    public void release() { available = true; }
    public String getSeatNo() { return seatNo; }
}
```

EXPLANATION:

All attributes in each class are declared private and accessed through public methods to ensure data integrity.For example, the Seat class encapsulates its seat number and availability to prevent direct modification from outside the class.

INHERITANCE (IS-A RELATIONSHIP)
DEFINITION:

Inheritance allows a subclass to reuse and extend properties and behaviors of a parent class.

In your system:

- The Payment class could be extended to represent different types of payments (like CreditCardPayment, UPIPayment, etc.).
- The Booking or Passenger classes could also be further extended.

Example (conceptual extension):

```
public class Payment {
    private int id;
    private double amount;
    private boolean confirmed;
    // common methods...
}
public class CreditCardPayment extends Payment {
    private String cardNumber;

    public CreditCardPayment(int id, double amount, String cardNumber) {
        super(id, amount);
        this.cardNumber = cardNumber;
    }
    // specialized behavior
```

```
}
```

EXPLANATION:

The system supports inheritance by allowing specialized classes (e.g., CreditCardPayment) to extend a general Payment class, promoting reusability and scalability.

AGGREGATION (HAS-A RELATIONSHIP)

DEFINITION:

Aggregation represents a relationship where one class contains another but can exist independently (a "has-a" relationship).

In your system:

- Flight has-a Aircraft
- Aircraft has-many Seats
- Booking has-a Passenger, Flight, and Payment

Example:

```
public class Flight {
   private Aircraft aircraft; // aggregation
}
public class Aircraft {
   private List<Seat> seats; // composition/aggregation
}
public class Booking {
   private Passenger passenger;
   private Flight flight;
   private Payment payment;
}
```

EXPLANATION:

Aggregation is used throughout the system:

Flight aggregates an Aircraft, while Booking aggregates Passenger, Flight, and Payment objects to form complete business logic relationships.

POLYMORPHISM (MANY FORMS / DYNAMIC BINDING)

DEFINITION:

Polymorphism allows a single interface to represent different underlying forms (method overriding or interface implementation).

In your system:

- BoardingPass uses a common method (printPass()) that could be overridden if you later extend it for different boarding types (like OnlineBoardingPass, CounterBoardingPass).
- Payment can be polymorphic if you later create multiple payment types (like CashPayment, CardPayment).

Example:

```
public class BoardingPass {
   public void printPass() {
      System.out.println("Standard Boarding Pass");
   }
}
public class OnlineBoardingPass extends BoardingPass {
   @Override
   public void printPass() {
      System.out.println("Digital Boarding Pass (QR Code)");
   }
}
```

Then, you can use:

```
BoardingPass bp = new OnlineBoardingPass();
```

```
bp.printPass(); // Calls overridden version dynamically
```

EXPLANATION:

Polymorphism is demonstrated by allowing methods like printPass() or confirm() to be overridden in subclasses to provide specialized behavior at runtime.

❖ IMPLEMENT THE MAIN METHOD FOR A MENU-DRIVEN SYSTEM:

PACKAGE : com.airline.main

MainApp.java

```
package com.airline.main;
import com.airline.model.*;
import com.airline.service.*;
import java.util.*;
public class MainApp {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      AirlineService service = new AirlineService();
      while (true) {
         System.out.println("\n===== Airline Reservation System =====");
         System.out.println("1. Add Flight");
         System.out.println("2. Add Passenger");
         System.out.println("3. Create Booking");
         System.out.println("4. Confirm Payment");
         System.out.println("5. Check-in & Issue Boarding Pass");
         System.out.println("6. Display Flights");
         System.out.println("7. Exit");
         System.out.print("Enter choice: ");
         int ch = sc.nextInt();
         sc.nextLine();
         switch (ch) {
            case 1 -> {
               System.out.print("Flight No: ");
               String no = sc.nextLine();
               System.out.print("Origin: ");
               String o = sc.nextLine();
               System.out.print("Destination: ");
               String d = sc.nextLine();
               System.out.print("Date/Time: ");
               String dt = sc.nextLine();
               Aircraft a = new Aircraft("A1", "Boeing 737", 5);
               service.addFlight(new Flight(no, o, d, dt, a));
            }
            case 2 -> {
               System.out.print("Passenger ID: ");
               int id = sc.nextInt();
               sc.nextLine();
               System.out.print("Name: ");
               String name = sc.nextLine();
               System.out.print("Contact: ");
               String c = sc.nextLine();
               service.addPassenger(new Passenger(id, name, c));
            }
            case 3 -> {
               System.out.print("Flight No: ");
```

```
                String fno = sc.nextLine();
                System.out.print("Passenger ID: ");
                int pid = sc.nextInt();
                System.out.print("Fare: ₹");
                double fare = sc.nextDouble();
                service.createBooking(fno, pid, fare);
            }
            case 4 -> {
                System.out.print("Booking ID to confirm: ");
                int bid = sc.nextInt();
                service.confirmPayment(bid);
            }
            case 5 -> {
                System.out.print("Booking ID to check-in: ");
                int bid = sc.nextInt();
                service.checkIn(bid);
            }
            case 6 -> service.displayFlights();
            case 7 -> {
                System.out.println("Exiting system...");
                System.exit(0);
            }
            default -> System.out.println("Invalid choice!");
        }
    }
  }
}
```

OUTPUT:

```
===== Airline Reservation System =====
1. Add Flight
2. Add Passenger
3. Create Booking
4. Confirm Payment
5. Check-in & Issue Boarding Pass
6. Display Flights
7. Exit
Enter choice: 1
Flight No: AI101
Origin: Delhi
Destination: Mumbai
Date/Time: 10-10-2025 10:00
Flight added successfully: AI101 | Delhi → Mumbai | 10-10-2025 10:00 | Seats: 5

===== Airline Reservation System =====
1. Add Flight
2. Add Passenger
3. Create Booking
4. Confirm Payment
5. Check-in & Issue Boarding Pass
6. Display Flights
7. Exit
Enter choice: 2
Passenger ID: 1
Name: Jeganath
Contact: 9363910641
Passenger added successfully: 1 - Jeganath (9363910641)

===== Airline Reservation System =====
1. Add Flight
2. Add Passenger
3. Create Booking
4. Confirm Payment
5. Check-in & Issue Boarding Pass
6. Display Flights
7. Exit
```

```
Enter choice: 3
Flight No: AI101
Passenger ID: 1
Fare: ₹4500
Booking created successfully!
Booking #1 | Jeganath | AI101 | Seat: S1
Payment ID: 1 | Amount: ₹4500.0 | Status: Pending

===== Airline Reservation System =====
1. Add Flight
2. Add Passenger
3. Create Booking
4. Confirm Payment
5. Check-in & Issue Boarding Pass
6. Display Flights
7. Exit
Enter choice: 4
Booking ID to confirm: 1
Payment confirmed for Booking #1

===== Airline Reservation System =====
1. Add Flight
2. Add Passenger
3. Create Booking
4. Confirm Payment
5. Check-in & Issue Boarding Pass
6. Display Flights
7. Exit
Enter choice: 5
Booking ID to check-in: 1

===== BOARDING PASS =====
Passenger: Jeganath
Flight: AI101
Seat: S1
Gate: A1
Status: CHECKED-IN
=========================


===== Airline Reservation System =====
1. Add Flight
2. Add Passenger
3. Create Booking
4. Confirm Payment
5. Check-in & Issue Boarding Pass
6. Display Flights
7. Exit
Enter choice: 7
Exiting system...
```

GITHUB LINK :

[jeganath047-ux/AirlineReservationCheck-inSystem]